

Лабораторна робота 1

Аналіз програмного коду мов високого рівня

1.1 Мета роботи

Отримати навички розпізнавання констукцій мов високого рівня в машинному кодї для архітектур x86/x64 та ARM/ARM64 на прикладї C/C++.

1.2 Постановка задачі

Дослідити машинний код, що відповідає синтаксичним конструкціям C/C++.

1.3 Порядок виконання роботи

Розглянемо процес генерації машинного коду сучасними компіляторами на прикладї простої програми мовою C, рекурсивний алгоритм розв'язку задачі про Ханойську вежу з трьома дисками:

```
$ cat hanoi.c
#include <stdio.h>

void hanoi(int n, char from, char to, char aux) {
    if (n == 1)
        printf("Move disk 1 from %c to %c\n", from, to);
    else {
        hanoi(n-1, from, aux, to);
        printf("Move disk %d from %c to %c\n", n, from, to);
        hanoi(n-1, aux, to, from);
    }
}

int main() {
    hanoi(3, 'A', 'C', 'B');
}

$ gcc hanoi.c && ./a.out
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

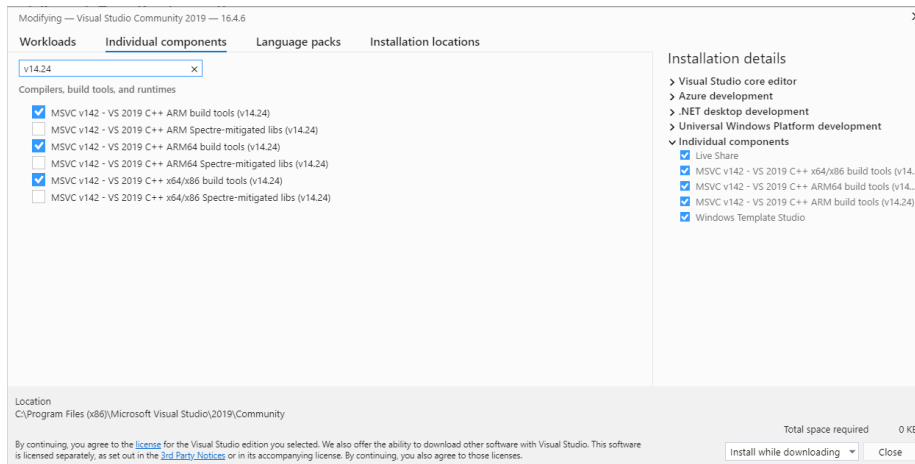


Рис. 1.1: Встановлені build tools для x86, x64, ARM, ARM64

1.3.1 Компілятор Microsoft C/C++ у Visual Studio 2019

Для Microsoft C++ використаємо утиліти командного рядка для різних платформ (x86, x64, ARM, ARM64). За замовчуванням вони не встановлюються, щоб додати в існуюче середовище Visual Studio 2019 необхідно виконати [20]. Приклад успішного налаштування на рис. 1.1.

Цільова архітектура задається змінними оточення у скрипті vcvarsall.bat, параметр arch x86, amd64, x86_arm, x86_arm64:

```
C:\Program Files (x86)\Microsoft Visual Studio\2019\Community\VC\Auxiliary\
Build>vcvarsall.bat /help
Syntax:
    vcvarsall.bat [arch] [platform_type] [winsdk_version] [-vcvars_ver=
        vc_version] [-vcvars_spectre_libs=spectre_mode]
where :
    [arch]: x86 | amd64 | x86_amd64 | x86_arm | x86_arm64 | amd64_x86 |
        amd64_arm | amd64_arm64
...

C:\Program Files (x86)\Microsoft Visual Studio\2019\Community\VC\Auxiliary\
Build>vcvarsall.bat x86_arm
*****
** Visual Studio 2019 Developer Command Prompt v16.4.6
** Copyright (c) 2019 Microsoft Corporation
*****
[vcvarsall.bat] Environment initialized for: 'x86_arm'
```

Компілятор cl.exe має параметр /FA, задавши який можна отримати лістинг з асемблерним та машинним кодом, що відповідає окремим рядкам вихідного коду мовою C [21]:

```
> cl /FAcsu hanoi.c
Microsoft (R) C/C++ Optimizing Compiler Version 19.24.28319 for x86
Copyright (C) Microsoft Corporation. All rights reserved.

hanoi.c
Microsoft (R) Incremental Linker Version 14.24.28319.0
Copyright (C) Microsoft Corporation. All rights reserved.

/out:hanoi.exe
hanoi.obj
```

В процесі роботи створюється файл hanoi.cod, фрагмент з функцією

hanoi має вигляд:

```

_hanoi PROC

; 3      : void hanoi(int n, char from, char to, char aux) {

    00000 55                push    ebp
    00001 8b ec             mov     ebp, esp

; 4      :      if (n == 1)

    00003 83 7d 08 01       cmp     DWORD PTR _n$[ebp], 1
    00007 75 19             jne     SHORT $LN2@hanoi

; 5      :      printf("Move disk 1 from %c to %c\n", from, to);

    00009 0f be 45 10       movsx   eax, BYTE PTR _to$[ebp]
    0000d 50                push    eax
    0000e 0f be 4d 0c       movsx   ecx, BYTE PTR _from$[ebp]
    00012 51                push    ecx
    00013 68 00 00 00 00    push    OFFSET $SG9159
    00018 e8 00 00 00 00    call    _printf
    0001d 83 c4 0c          add     esp, 12
    00020 eb 57             jmp     SHORT $LN1@hanoi
$LN2@hanoi:

; 6      :      else {
; 7      :      hanoi(n-1, from, aux, to);

    00022 0f b6 55 10       movzx   edx, BYTE PTR _to$[ebp]
    00026 52                push    edx
    00027 0f b6 45 14       movzx   eax, BYTE PTR _aux$[ebp]
    0002b 50                push    eax
    0002c 0f b6 4d 0c       movzx   ecx, BYTE PTR _from$[ebp]
    00030 51                push    ecx
    00031 8b 55 08          mov     edx, DWORD PTR _n$[ebp]
    00034 83 ea 01          sub     edx, 1
    00037 52                push    edx
    00038 e8 00 00 00 00    call    _hanoi
    0003d 83 c4 10          add     esp, 16

; 8      :      printf("Move disk %d from %c to %c\n", n, from, to);

    00040 0f be 45 10       movsx   eax, BYTE PTR _to$[ebp]
    00044 50                push    eax
    00045 0f be 4d 0c       movsx   ecx, BYTE PTR _from$[ebp]
    00049 51                push    ecx
    0004a 8b 55 08          mov     edx, DWORD PTR _n$[ebp]
    0004d 52                push    edx
    0004e 68 00 00 00 00    push    OFFSET $SG9160
    00053 e8 00 00 00 00    call    _printf
    00058 83 c4 10          add     esp, 16

; 9      :      hanoi(n-1, aux, to, from);

    0005b 0f b6 45 0c       movzx   eax, BYTE PTR _from$[ebp]
    0005f 50                push    eax
    00060 0f b6 4d 10       movzx   ecx, BYTE PTR _to$[ebp]
    00064 51                push    ecx
    00065 0f b6 55 14       movzx   edx, BYTE PTR _aux$[ebp]
    00069 52                push    edx
    0006a 8b 45 08          mov     eax, DWORD PTR _n$[ebp]
    0006d 83 e8 01          sub     eax, 1
    00070 50                push    eax
    00071 e8 00 00 00 00    call    _hanoi
    00076 83 c4 10          add     esp, 16
$LN1@hanoi:

; 10     :      }
; 11     :  }

    00079 5d                pop     ebp
    0007a c3                ret     0
_hanoi ENDP

```

Зверніть увагу на передачу параметрів функцій, послідовність та механізм (стек, регістри та їхній порядок) відрізняються для різних платформ та ОС [22].

1.3.2 Компілятор gcc у GCC 9.2

Розглянемо набір компіляторів GNU в ОС Linux для того ж набору архітектур (в GCC позначаються i686, amd64, arm, aarch64). Для інсталяції відповідних компіляторів та утиліт в Kali необхідно виконати:

```
# apt install gcc gcc-i686-linux-gnu gcc-arm-linux-gnueabi gcc-aarch64-linux-  
gnu binutils binutils-arm-linux-gnueabi binutils-aarch64-linux-gnu
```

Параметри компілятора gcc для виводу асемблерного листінгу -Wa,-adhln -g [23]:

```
$ gcc -Wa,-adhln -g hanoi.c > hanoi.amd64.lst  
$ i686-linux-gnu-gcc -Wa,-adhln -g hanoi.c > hanoi.i686.lst  
$ arm-linux-gnueabi-gcc -Wa,-adhln -g hanoi.c > hanoi.arm.lst  
$ aarch64-linux-gnu-gcc -Wa,-adhln -g hanoi.c > hanoi.aarch64.lst
```

В результаті для i686 фрагмент коду має вигляд:

```
1:hanoi.c      **** #include <stdio.h>  
2:hanoi.c      ****  
3:hanoi.c      **** void hanoi(int n, char from, char to, char aux) {  
15              .loc 1 3 49  
16              .cfi_startproc  
17 0000 55      pushl    %ebp  
18              .cfi_def_cfa_offset 8  
19              .cfi_offset 5, -8  
20 0001 89E5     movl     %esp, %ebp  
21              .cfi_def_cfa_register 5  
22 0003 56      pushl    %esi  
23 0004 53      pushl    %ebx  
24 0005 83EC10   subl     $16, %esp  
25              .cfi_offset 6, -12  
26              .cfi_offset 3, -16  
27 0008 E8FCFFFF call     __x86.get_pc_thunk.bx  
27              FF  
28 000d 81C30200 addl     $_GLOBAL_OFFSET_TABLE_, %ebx  
28              0000  
29 0013 8B4D0C   movl     12(%ebp), %ecx  
30 0016 8B5510   movl     16(%ebp), %edx  
31 0019 8B4514   movl     20(%ebp), %eax  
32 001c 884DF4   movb     %cl, -12(%ebp)  
33 001f 8855F0   movb     %dl, -16(%ebp)  
34 0022 8845EC   movb     %al, -20(%ebp)  
4:hanoi.c      **** if (n == 1)  
35              .loc 1 4 8  
36 0025 837D0801 cmpl     $1, 8(%ebp)  
37 0029 751E     jne      .L2  
5:hanoi.c      **** printf("Move disk 1 from %c to %c\n", from,  
to);  
38              .loc 1 5 9  
39 002b 0FB555F0 movsbl   -16(%ebp), %edx  
40 002f 0FB545F4 movsbl   -12(%ebp), %eax  
41 0033 83EC04   subl     $4, %esp  
42 0036 52      pushl    %edx  
43 0037 50      pushl    %eax  
44 0038 8D830000 leal     .LC0@GOTOFF(%ebx), %eax  
44              0000  
45 003e 50      pushl    %eax  
46 003f E8FCFFFF call     printf@PLT  
46              FF  
47 0044 83C410   addl     $16, %esp  
6:hanoi.c      **** else {  
7:hanoi.c      **** hanoi(n-1, from, aux, to);  
8:hanoi.c      **** printf("Move disk %d from %c to %c\n", n,  
from, to);
```

```

9:hanoi.c      ****      hanoi(n-1, aux, to, from);
10:hanoi.c     ****      }
11:hanoi.c     **** }
48
49 0047 EB58          .loc 1 11 1
                    jmp      .L4
50
51               .L2:      printf("Move disk %d from %c to %c\n", n,
                    from, to);
52
53 0049 0FBE4DF0      .loc 1 7 9
                    movsbl   -16(%ebp), %ecx
54 004d 0FBE55EC      movsbl   -20(%ebp), %edx
55 0051 0FBE45F4      movsbl   -12(%ebp), %eax
56 0055 8B7508      movl      8(%ebp), %esi
57 0058 83EE01      subl      $1, %esi
58 005b 51          pushl     %ecx
59 005c 52          pushl     %edx
60 005d 50          pushl     %eax
61 005e 56          pushl     %esi
62 005f E8FCFFFF      call      hanoi
63 0064 83C410      addl      $16, %esp
8:hanoi.c      ****      hanoi(n-1, aux, to, from);
63
64 0067 0FBE55F0      .loc 1 8 9
                    movsbl   -16(%ebp), %edx
65 006b 0FBE45F4      movsbl   -12(%ebp), %eax
66 006f 52          pushl     %edx
67 0070 50          pushl     %eax
68 0071 FF7508      pushl     8(%ebp)
69 0074 8D831B00      leal     .LC1@GOTOFF(%ebx), %eax
70 007a 50          pushl     %eax
71 007b E8FCFFFF      call      printf@PLT
72 0080 83C410      addl      $16, %esp
9:hanoi.c      ****      }
73
74 0083 0FBE4DF4      .loc 1 9 9
                    movsbl   -12(%ebp), %ecx
75 0087 0FBE55F0      movsbl   -16(%ebp), %edx
76 008b 0FBE45EC      movsbl   -20(%ebp), %eax
77 008f 8B5D08      movl      8(%ebp), %ebx
78 0092 83EB01      subl      $1, %ebx
79 0095 51          pushl     %ecx
80 0096 52          pushl     %edx
81 0097 50          pushl     %eax
82 0098 53          pushl     %ebx
83 0099 E8FCFFFF      call      hanoi
84 009e 83C410      addl      $16, %esp
85
86               .L4:      .loc 1 11 1
87 00a1 90          nop
88 00a2 8D65F8      leal     -8(%ebp), %esp
89 00a5 5B          popl      %ebx
90               .cfi_restore 3
91 00a6 5E          popl      %esi
92               .cfi_restore 6
93 00a7 5D          popl      %ebp
94               .cfi_restore 5
95               .cfi_def_cfa 4, 4
96 00a8 C3          ret

```

Зверніть увагу на несхожість лістингу з виводом MSVC cl. У GCC більш поширений так званий AT&T синтаксис, проти Intel у MSVS [5]. Отримати лістинг з вихідним та відповідним машинним кодом можна і іншим шляхом, дизасемблювавши бінарний виконуваний файл створений з відлагоджувальною інформацією [24]:

```

$ i686-linux-gnu-gcc -g hanoi.c -o hanoi.i686
$ i686-linux-gnu-objdump -drwC -Mintel -S hanoi.i686 > hanoi.i686.2.lst

```

Відповідний фрагмент з Intel синтаксисом:

```

000011ed <hanoi>:
#include <stdio.h>

void hanoi(int n, char from, char to, char aux) {
11ed:      55                push    ebp
11ee:      89 e5             mov     ebp,esp
11f0:      56                push    esi
11f1:      53                push    ebx
11f2:      83 ec 10           sub     esp,0x10
11f5:      e8 b6 fe ff ff     call    10b0 <_x86.get_pc_thunk.bx>
11fa:      81 c3 06 2e 00 00    add     ebx,0x2e06
1200:      8b 4d 0c             mov     ecx,DWORD PTR [ebp+0xc]
1203:      8b 55 10             mov     edx,DWORD PTR [ebp+0x10]
1206:      8b 45 14             mov     eax,DWORD PTR [ebp+0x14]
1209:      88 4d f4             mov     BYTE PTR [ebp-0xc],cl
120c:      88 55 f0             mov     BYTE PTR [ebp-0x10],dl
120f:      88 45 ec             mov     BYTE PTR [ebp-0x14],al
if (n == 1)
1212:      83 7d 08 01          cmp     DWORD PTR [ebp+0x8],0x1
1216:      75 1e                jne     1236 <hanoi+0xa9>
    printf("Move disk 1 from %c to %c\n", from, to);
1218:      0f be 55 f0          movsx   edx,BYTE PTR [ebp-0x10]
121c:      0f be 45 f4          movsx   eax,BYTE PTR [ebp-0xc]
1220:      83 ec 04             sub     esp,0x4
1223:      52                push    edx
1224:      50                push    eax
1225:      8d 83 08 e0 ff ff     lea     eax,[ebx-0x1ff8]
122b:      50                push    eax
122c:      e8 0f fe ff ff     call    1040 <printf@plt>
1231:      83 c4 10             add     esp,0x10
    else {
        hanoi(n-1, from, aux, to);
        printf("Move disk %d from %c to %c\n", n, from, to);
        hanoi(n-1, aux, to, from);
    }
}

1234:      eb 58                jmp     128e <hanoi+0xa1>
    hanoi(n-1, from, aux, to);
1236:      0f be 4d f0          movsx   ecx,BYTE PTR [ebp-0x10]
123a:      0f be 55 ec          movsx   edx,BYTE PTR [ebp-0x14]
123e:      0f be 45 f4          movsx   eax,BYTE PTR [ebp-0xc]
1242:      8b 75 08             mov     esi,DWORD PTR [ebp+0x8]
1245:      83 ee 01             sub     esi,0x1
1248:      51                push    ecx
1249:      52                push    edx
124a:      50                push    eax
124b:      56                push    esi
124c:      e8 9c ff ff ff     call    11ed <hanoi>
1251:      83 c4 10             add     esp,0x10
    printf("Move disk %d from %c to %c\n", n, from, to);
1254:      0f be 55 f0          movsx   edx,BYTE PTR [ebp-0x10]
1258:      0f be 45 f4          movsx   eax,BYTE PTR [ebp-0xc]
125c:      52                push    edx
125d:      50                push    eax
125e:      ff 75 08             push    DWORD PTR [ebp+0x8]
1261:      8d 83 23 e0 ff ff     lea     eax,[ebx-0x1fdd]
1267:      50                push    eax
1268:      e8 d3 fd ff ff     call    1040 <printf@plt>
126d:      83 c4 10             add     esp,0x10
    hanoi(n-1, aux, to, from);
1270:      0f be 4d f4          movsx   ecx,BYTE PTR [ebp-0xc]
1274:      0f be 55 f0          movsx   edx,BYTE PTR [ebp-0x10]
1278:      0f be 45 ec          movsx   eax,BYTE PTR [ebp-0x14]
127c:      8b 5d 08             mov     ebx,DWORD PTR [ebp+0x8]
127f:      83 eb 01             sub     ebx,0x1
1282:      51                push    ecx
1283:      52                push    edx
1284:      50                push    eax
1285:      53                push    ebx
1286:      e8 62 ff ff ff     call    11ed <hanoi>
128b:      83 c4 10             add     esp,0x10
}

128e:      90                nop
128f:      8d 65 f8             lea     esp,[ebp-0x8]

```

1292:	5b	pop	ebx
1293:	5e	pop	esi
1294:	5d	pop	ebp
1295:	c3	ret	

Порівнявши отримані результати з MSVC cl, можна побачити, що вони також різні. Це штатна ситуація, що ілюструє чому відновлення вихідного коду за бінарним виконуваним файлом є складною задачею – результуючий машинний код в більшості випадків різний як для різних компіляторів, так і для різних версій одного компілятора, різних налаштувань оптимізації та ін.

В подальшому для спрощення налаштування програм для різних архітектур корисно додати прозору підтримку не-x86 за допомогою qemu-user-binfmt:

```
# dpkg --add-architecture armel
# dpkg --add-architecture arm64
# apt update
# apt install qemu-user-binfmt libc6:arm64 libc6:armel
```

У разі успіху з'являється можливість прямого запуску виконуваних файлів для інших платформ, наприклад AArch64:

```
$ aarch64-linux-gnu-gcc hanoi.c && file a.out && ./a.out
a.out: ELF 64-bit LSB shared object, ARM aarch64, version 1 (SYSV),
dynamically linked, interpreter /lib/ld-linux-aarch64.so.1, BuildID[sha1
]=71a42dbc03e5537d0208f602435bd16b56cc7109, for GNU/Linux 3.7.0, not
stripped
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Більш детально з особливостями архітектур x86/amd64 та ARM/AArch64, наборами інструкцій, відповідностями конструкцій C/C++ до машинного коду можна ознайомитися у [5].

1.4 Варіанти завдань

- Проаналізувати машинний код прикладу hanoi.c для Windows x64, ARM, ARM64 (MSVC), для Linux amd64, arm, arm64 (GCC), для Linux amd64 (LLVM clang, <https://llvm.org/>);
- Реалізувати мовою C/C++, проаналізувати результати компіляції (за варіантом), для платформ i686, amd64, arm, aarch64:

– Комбінаторні алгоритми [25], будь-який на Ваш вибір з вказаного класу:

1. на графах – обхід графа;
2. на графах – топологічне сортування;
3. на графах – компонента зв'язності графа;
4. на графах – побудова кістякового дерева;
5. на графах – пошук найкоротшого шляху;
6. на графах – розфарбування графів;

7. на графах – пошук найвигіднішого шляху;
 8. на графах – потоки в мережах;
 9. на графах – клік;
 10. на графах – цикли;
 11. на графах – паросполучення;
 12. на графах – ізоморфізми;
 13. пошуку в масиві – елементи впорядковані;
 14. пошуку в масиві – елементи не впорядковані;
 15. пошуку в рядках – приблизний збіг;
 16. сортування – обміном;
 17. сортування – вибором;
 18. сортування – включенням;
 19. сортування – злиттям;
 20. сортування – без порівнянь.
- Криптографічні алгоритми, алгоритми кодування та контролю цілісності [26, 27]:
1. AES;
 2. DES;
 3. IDEA;
 4. CAST5;
 5. Blowfish;
 6. ARCfour / RC4;
 7. SEED;
 8. Serpent;
 9. Camellia;
 10. Salsa20;
 11. ChaCha20;
 12. GOST 28147-89;
 13. MD2;
 14. MD4;
 15. MD5;
 16. SHA-1;
 17. SHA-224;
 18. SHA-256;
 19. SHA-384;
 20. SHA3-256;
 21. SHAKE256;
 22. RIPEMD-160;
 23. Whirlpool;
 24. CRC-32;
 25. GOST R 34.11-2012 (Stribog);
 26. BLAKE2b;
 27. RC5;

28. XXTEA;
29. Raiden;
30. VMPC.

- Реалізація функцій стандартної бібліотеки C. Бібліотека за варіантами, функції всі зазначені (за наявності реалізації), версія бібліотеки остання стабільна на момент початку курсу:

1. glibc [28];
2. dietlibc [29];
3. uClibc-ng [30];
4. Newlib [31];
5. musl [32];
6. klibc [33];
7. bionic [34].

Функції:

- стандартного ввід-виводу `printf`, `puts`;
- роботи з файлами `fopen`, `fread`, `fwrite`, `feof`, `fclose`;
- виконання команд операційної системи `system`.

Зверніть увагу на відмінності системних викликів у різних ОС Linux та Windows для різних архітектур (x86, x86_64, ARM) [35].

1.5 Контрольні питання

1. Як на рівні машинного коду реалізовано `switch-case` у розглянутих компіляторах?
2. Що відбувається із файловими дескрипторами батьківського процесу у виклику `system` для `suid` та `ne-suid` виконуваних файлів? Розгляньте випадок, коли дескриптори 0, 1, 2 у батьківському процесі закриті.
3. Що відбувається з змінними оточення, змінними та експортованими функціями командної оболонки (`bash`) у виклику `system` для `suid` та `ne-suid` виконуваних файлів?